

While I personally think the following is a bad idea, it demonstrates the ability to add an instance variable to an object dynamically:

```
myObject = MyClass()
myObject.someInstanceVariable = 5
```

In order to allow for this dynamic capability, dictionaries are typically implemented starting with enough empty space to represent some number of instance variables (the exact number is an internal detail of Python). Whenever a new instance variable is encountered, it's added to the dictionary. If the dictionary runs out of space, Python adds more. This generally works well, and programmers do not experience any problems with this implementation.

However, imagine you have a class like the following with two instance variables created in the `__init__()` method, and you know that you will not need to add any more instance variables:

```
class Point():
    def __init__(self, x, y):
        self.x = x
        self.y = y
    # More methods
```

Now, let's assume that you need to instantiate a very large number (hundreds of thousands, or even millions) of objects from this class. A case like this could cumulatively account for a large amount of wasted memory space (RAM).

To combat this potential waste, Python gives us a different approach, known as *slots*, to represent the instance variables. The idea is that you can tell Python the names of all the instance variables up front, and Python will use a data structure that allocates exactly enough space for just those instance variables. To use slots, you need to include the special class variable `__slots__` to define a list of variables:

```
__slots__ = [<instanceVar1>, <instanceVar2>, ... <instanceVarN>]
```

Here is what a modified version of our example class would look like: